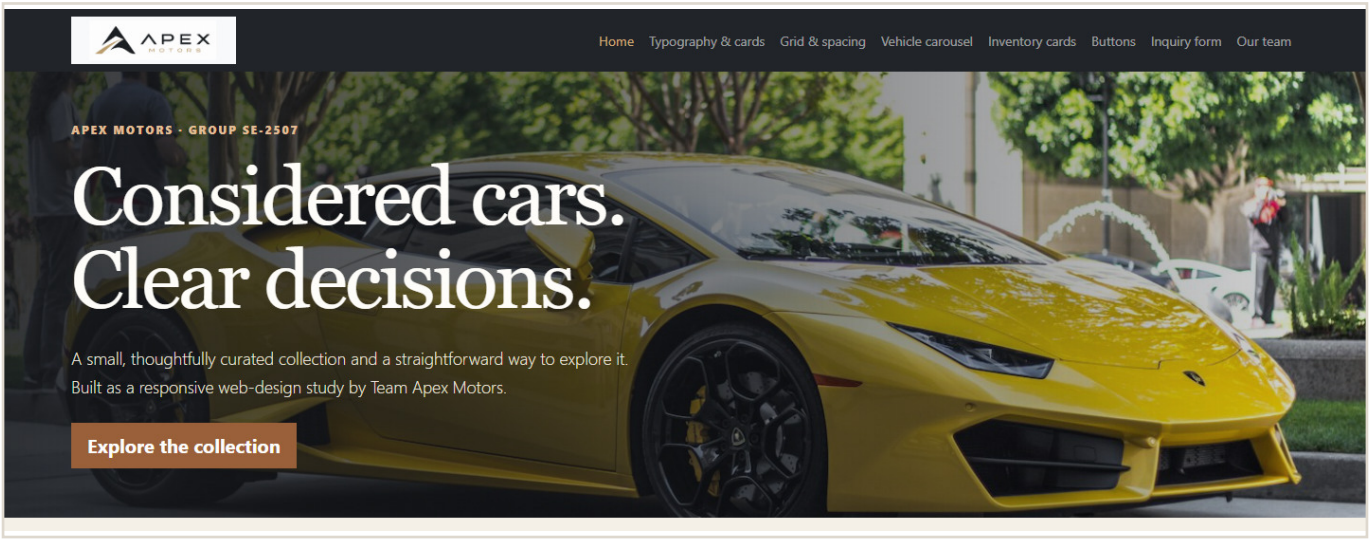
WEB TECHNOLOGIES · ASSIGNMENT 5

JavaScript and DOM Manipulation.

Continue the Assignment 3 website with DOM updates, event handling, objects, arrays, callbacks, sound and animation.



The updated home page keeps the original Bootstrap layout and vehicle hero.

Team: Apex Motors Group: SE-2507
Members: Arsen, Kuttibay, Farabi, Muhammad
Prepared: 28 October 2026
Build: Bootstrap 5.3.3 and separate HTML, CSS and JavaScript

Objective. Add the required JavaScript features to the team's existing eight-page dealership site while retaining its navigation, colours, components and photographs.

Website: https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/
Repository: https://github.com/Vertex-blip/Car-dealership-website



Arsen



Kuttibay



Farabi

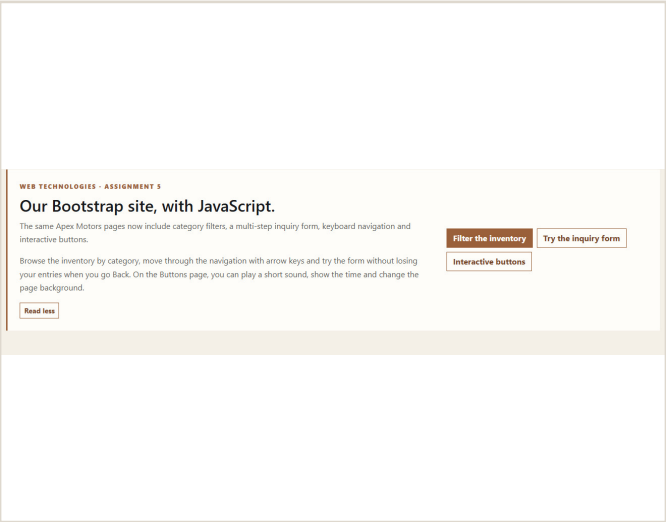


Muhammad

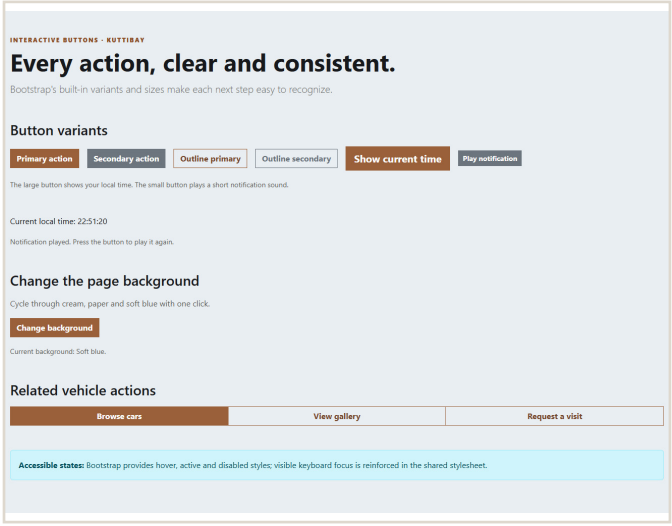
DOM updates and styling

The brief offers several alternatives. This project uses Read More for content visibility and a direct background-colour change for dynamic styling.

1. Select the Read More button and extra text; attach a click listener.
2. Change style.display and the button state to reveal or hide the extra content.
3. Cycle a small array of colours and assign the selected colour to body.style.backgroundColor.



Home: additional content revealed.



Buttons: background changed by JavaScript.

js/site.js

```
 71    const readMoreButton = document.getElementById('readMoreButton');
 72    const readMoreContent = document.getElementById('readMoreContent');
 73    if (readMoreButton && readMoreContent) {
 74      // The selected attribute task changes style.display without rebuilding the text.
 75      readMoreContent.hidden = false;
 76      readMoreContent.style.display = 'none';
 77      readMoreButton.setAttribute('aria-expanded', 'false');
 78      readMoreButton.addEventListener('click', () => {
 79        const expanded = readMoreButton.getAttribute('aria-expanded') !== 'true';
```

js/site.js

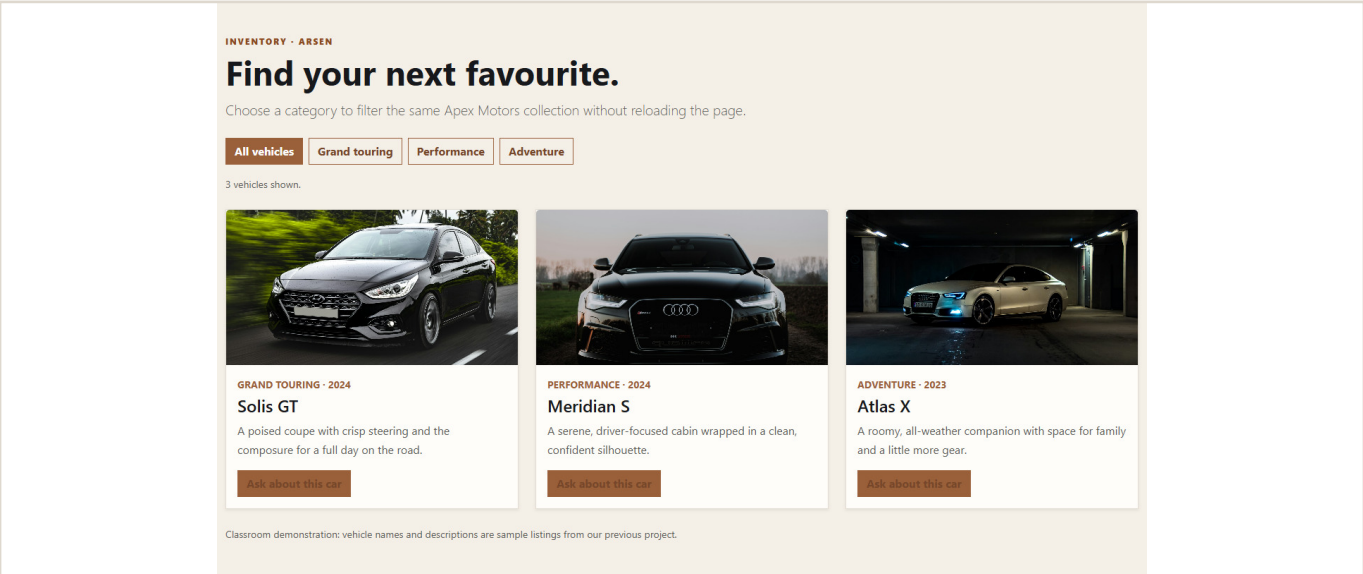
```
 96      backgroundStatus.textContent = `Current background: ${colors[currentColor].name}.`;
 97      backgroundButton.addEventListener('click', () => {
 98        currentColor = (currentColor + 1) % colors.length;
 99        const color = colors[currentColor];
100        document.body.style.backgroundColor = color.value;
101        backgroundStatus.textContent = `Current background: ${color.name}.`;
102      });
103    }
```

Actual source: js/site.js

Objects, arrays and vehicle cards

The three existing inventory cards are generated from structured vehicle data. The Bootstrap card layout and original photographs are retained.

1. Store each vehicle's name, category, photo and description in an object with a label method.
2. Keep the vehicle objects in an array and create DOM cards in an explicit for...of loop.
3. Read object properties and insert content with textContent rather than user-written HTML.



Inventory: the original three cards, now rendered from objects.

js/site.js

```
  6       return {
  7         id, name, category, categoryLabel, year, image, alt, description,
  8         label() {
  9           return `${this.categoryLabel} · ${this.year}`;
 10         }
 11       };
```

js/site.js

```
 15     const vehicles = [
 16       createVehicle('solis', 'Solis GT', 'touring', 'Grand touring', 2024,
 17         'images/car-01.jpg', 'Black sedan driving along a tree-lined road',
 18         'A poised coupe with crisp steering and the composure for a full day on the road.'),
 19       createVehicle('meridian', 'Meridian S', 'performance', 'Performance', 2024,
```

js/inventory.js

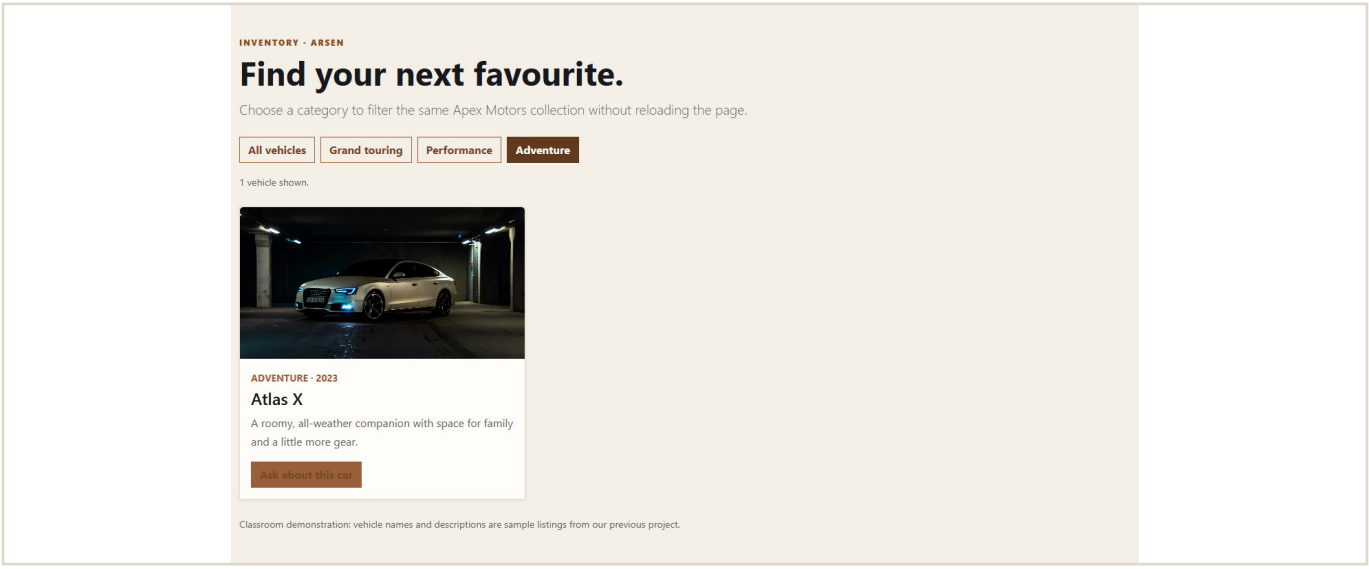
```
 52       const selectedVehicles = vehicles.filter(matchesCategory);
 53       const fragment = document.createDocumentFragment();
 54       for (const vehicle of selectedVehicles) fragment.append(makeCard(vehicle));
 55       grid.replaceChildren(fragment);
 56       count.textContent = `${selectedVehicles.length} ${selectedVehicles.length === 1 ?
     'vehicle' : 'vehicles'} shown.`;
 57       if (animate) window.Apex.animate(grid);
```

Actual source: js/site.js, js/inventory.js

Category filtering

Category buttons filter the existing collection without a page reload. The result count and visible cards respond to the selected category.

1. Listen for category button clicks and read the selected category.
2. Use a switch statement to choose the matching category condition.
3. Pass that condition to filter, then render the resulting array and update the result count.



Inventory: a selected category updates the collection.

js/inventory.js

```
19    function matchesCategory(vehicle) {
20      // Each switch branch controls which objects reach the visible card grid.
21      switch (category) {
22        case 'touring': return vehicle.category === 'touring';
23        case 'performance': return vehicle.category === 'performance';
24        case 'adventure': return vehicle.category === 'adventure';
25        default: return true;
26      }
27    }
28
29    function makeCard(vehicle) {
30      const column = element('div', 'col');
```

js/inventory.js

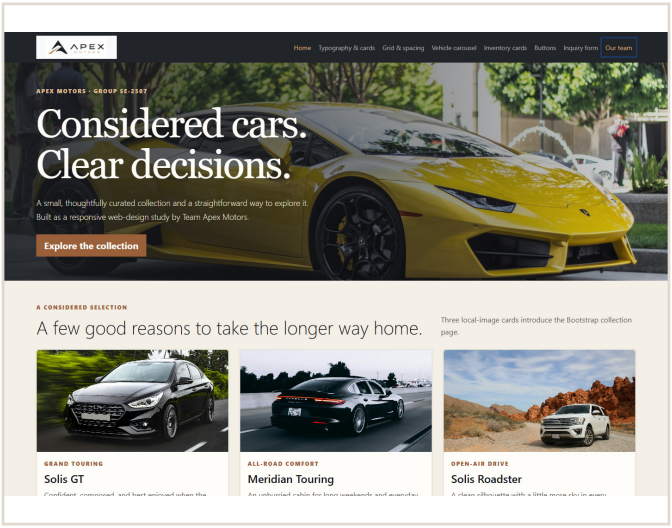
```
50    function render(animate = true) {
51      // filter is higher-order: it receives matchesCategory as its predicate callback.
52      const selectedVehicles = vehicles.filter(matchesCategory);
53      const fragment = document.createDocumentFragment();
54      for (const vehicle of selectedVehicles) fragment.append(makeCard(vehicle));
55      grid.replaceChildren(fragment);
56      count.textContent = `${selectedVehicles.length} ${selectedVehicles.length === 1 ?
    'vehicle' : 'vehicles'} shown.`;
57      if (animate) window.Apex.animate(grid);
```

Actual source: js/inventory.js

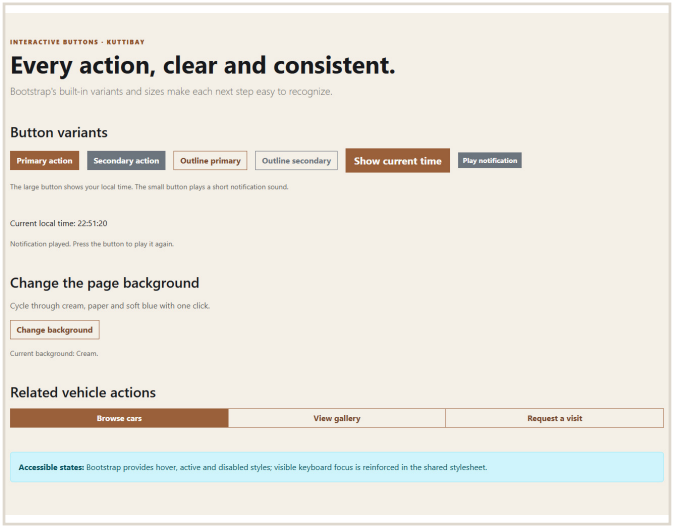
Button and keyboard events

The shared eight-link menu keeps normal link behaviour and adds the required arrow-key focus movement. Buttons also update the page through event listeners.

1. Select the navigation links and register a keydown listener on the menu.
2. Detect arrow keys, calculate the next or previous index with wraparound, and focus that link.
3. Use the time button's click callback to show a newly formatted current time.



A menu link receives keyboard focus.



Buttons: working time and sound actions.

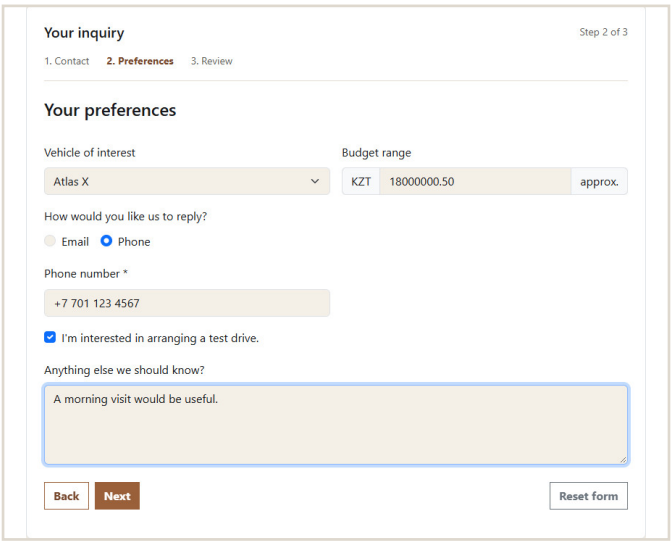
```
js/site.js
 45      navigation.addEventListener('keydown', event => {
 46        const current = links.indexOf(document.activeElement);
 47        if (current === -1) return;
 48        let next;
 49        switch (event.key) {
 50          case 'ArrowRight':
 51          case 'ArrowDown': next = (current + 1) % links.length; break;
 52          case 'ArrowLeft':
 53          case 'ArrowUp': next = (current - 1 + links.length) % links.length; break;
 54          case 'Home': next = 0; break;
 55          case 'End': next = links.length - 1; break;
js/site.js
 66        event.preventDefault();
 67        links[next].focus();
 68      });
js/site.js
107    if (timeButton && timeOutput) {
108      timeButton.addEventListener('click', () => {
109        timeOutput.textContent = `Current local time: ${new Date().toLocaleTimeString('en-
     GB')}`;
110        window.Apex.animate(timeOutput);
111      });
```

Actual source: js/site.js

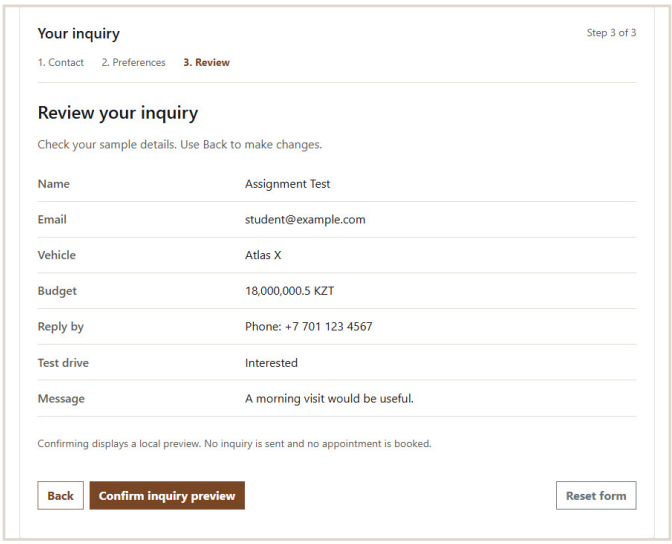
One inquiry, three steps

The multi-step form is the chosen callback option. The original inquiry fields are grouped into contact details, preferences, and review.

1. Keep the current step in state and validate it before moving forward.
2. Pass the step-display function as a callback to the navigation function.
3. Hide or reveal fieldsets without recreating their inputs, so Back retains entered values.



Step 2: inquiry preferences.



Inquiry: review before confirmation.

js/booking.js

```
135    function navigateStep(direction, onStepChange) {
136      if (direction > 0 && !validateStep(currentStep)) {
137        status.textContent = 'Please correct the highlighted fields before continuing.';
138        return;
139      }
140      const nextStep = Math.max(0, Math.min(2, currentStep + direction));
141      onStepChange(nextStep);
142    }
143    nextButton.addEventListener('click', () => navigateStep(1, showStep));
144    previousButton.addEventListener('click', () => navigateStep(-1, showStep));
145
```

js/booking.js

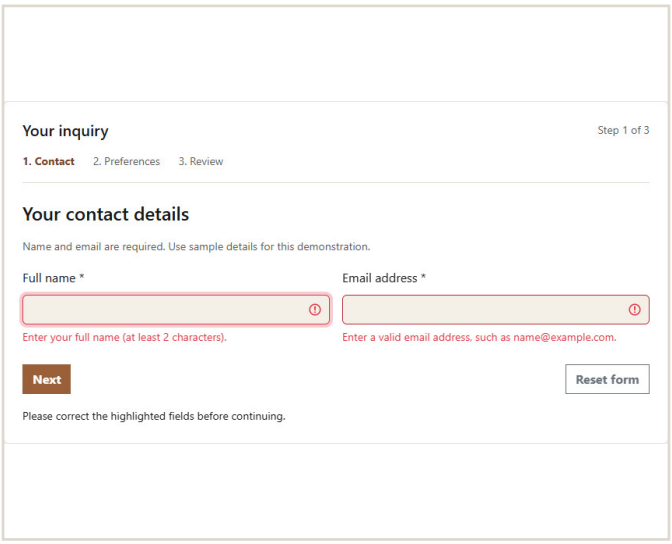
```
108    function showStep(index, moveFocus = true) {
109      currentStep = index;
110      steps.forEach((step, position) => { step.hidden = position !== currentStep; });
111      previousButton.hidden = currentStep === 0;
112      nextButton.hidden = currentStep === 2;
113      submitButton.hidden = currentStep !== 2;
114      document.getElementById('inquiryStepCount').textContent = `Step ${currentStep + 1} of
     3`;
115      document.querySelectorAll('[data-step-label]').forEach((label, position) => {
116        if (position === currentStep) label.setAttribute('aria-current', 'step');
117        else label.removeAttribute('aria-current');
```

Actual source: js/booking.js

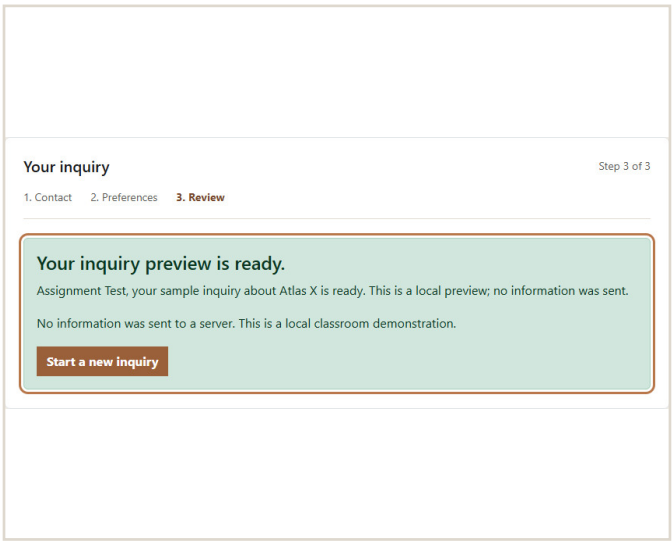
Validation, review and reset

The form checks required contact values and the user's preferences, then displays a local preview result. It does not send personal information to a server.

1. Prevent default submission, display field errors, and focus the first invalid control.
2. Build the review from the entered values and show confirmation without reloading.
3. Reset all fields, error messages, and step state when the user starts again.



Missing or invalid values receive clear feedback.



A completed inquiry preview remains on the page.

js/booking.js

```
 66    function validateStep(index, focusFirst = true) {
 67      let firstInvalid = null;
 68      for (const id of fieldsByStep[index]) {
 69        if (!setError(id, fieldError(id)) && !firstInvalid) firstInvalid =
    document.getElementById(id);
 70      }
 71      if (focusFirst && firstInvalid) firstInvalid.focus();
 72      return !firstInvalid;
 73    }
 74
 75    function clearResolvedStatus() {
 76      // Read validity without adding new error messages before a navigation attempt.
```

js/booking.js

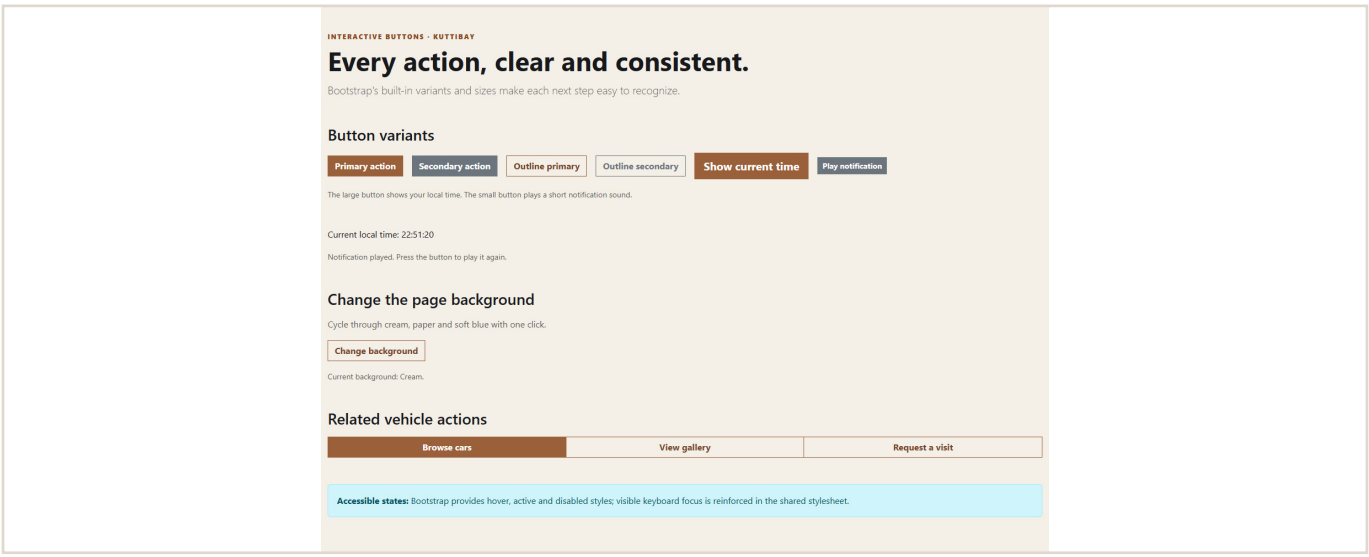
```
146    function clearValues() {
147      form.reset();
148      form.querySelectorAll('input, textarea').forEach(input => {
149        if (input.type === 'radio') input.checked = input.id === 'replyEmail';
150        else if (input.type === 'checkbox') input.checked = false;
151        else input.value = '';
152      });
153      vehicleSelect.value = '';
154      fieldsByStep.flat().forEach(id => setError(id, ''));
155      document.getElementById('inquirySummary').replaceChildren();
```

Actual source: js/booking.js

Sound and event-triggered animation

Sound and motion are attached to intentional user actions. The sound file is included locally and playback is reused instead of creating overlapping clips.

1. Load one Audio player and start it only after the notification button is clicked.
2. Pause and rewind the player before each playback; report playback failures clearly.
3. Animate content, inventory or form changes and respect the reduced-motion preference.



Buttons: user-triggered notification sound and feedback.

js/site.js

```
118      const sound = new Audio('assets/chime.wav');
119      sound.preload = 'auto';
120      sound.volume = 0.4;
121      let requestNumber = 0;
122      async function playChime() {
123        const currentRequest = ++requestNumber;
124        sound.pause();
125        sound.currentTime = 0;
126        try {
127          await sound.play();
128          if (currentRequest !== requestNumber) return;
129          soundStatus.textContent = 'Playing the notification sound.';
```

js/site.js

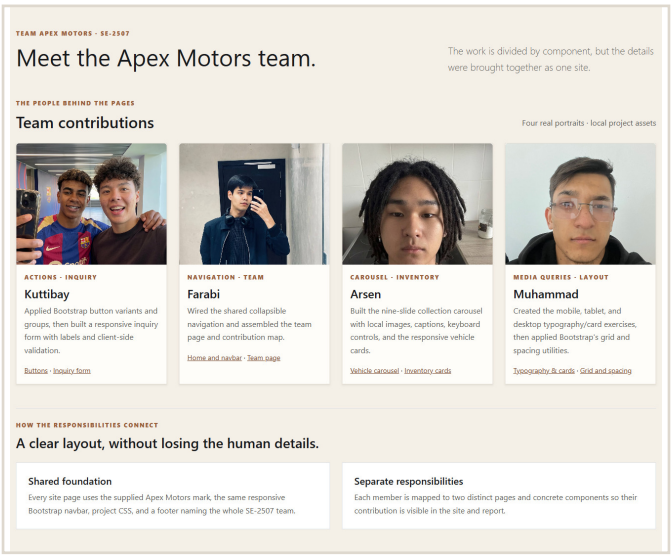
```
 30      animate(element) {
 31        if (!element || !element.animate
 32            || window.matchMedia('(prefers-reduced-motion: reduce)').matches) return;
 33        animations.get(element)?.cancel();
 34        const animation = element.animate(
 35          [{opacity: 0, transform: 'translateY(10px)'}, {opacity: 1, transform:
    'translateY(0)'}],
 36          {duration: 260, easing: 'ease-out'}
 37        );
 38        animations.set(element, animation);
```

Actual source: js/site.js

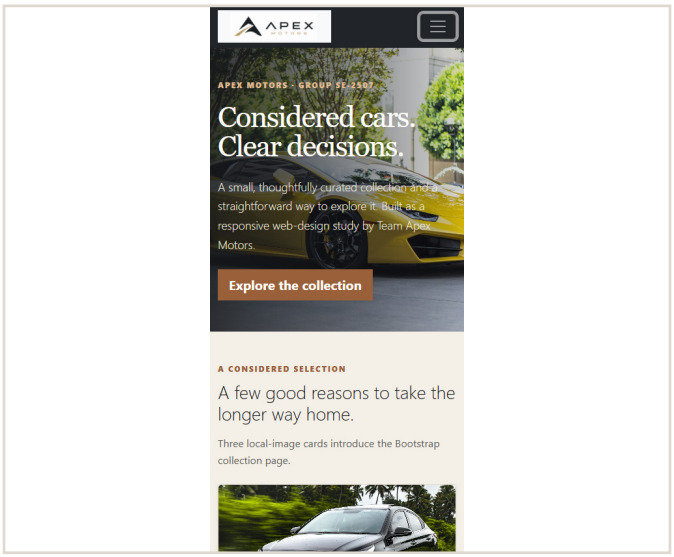
The same eight-page website

HTML, CSS and JavaScript are kept separate. The original responsive pages, team photos, card layout, carousel, colours and shared navigation continue from Assignment 3.

1. Load the local Bootstrap files, shared stylesheet and external deferred scripts.
2. Retain labels, alternative text, visible focus, a skip link and the mobile collapse menu.
3. Check all eight pages and the four original team portraits at desktop and phone widths.



The same four team members and portraits.



Original responsive layout at phone width.

index.html

```
11      <link href="vendor/bootstrap.min.css" rel="stylesheet" />
12      <link href="styles.css?v=bootstrap-a5" rel="stylesheet" />
13      <link href="enhancements.css?v=bootstrap-a5" rel="stylesheet" />
14      <script src="vendor/bootstrap.bundle.min.js" defer></script>
15      <script src="js/site.js?v=bootstrap-a5" defer></script>
16      <link rel="icon" href="images/apex-motors-logo.png" />
17    </head>
18    <body>
```

styles.css

```
1  :root {
2    --apex-ink: #17191c;
3    --apex-navy: #1d2b3a;
4    --apex-copper: #9a603a;
5    --apex-cream: #f5f0e7;
6    --apex-paper: #fffdf9;
7    --apex-muted: #5c5c59;
8    --bs-primary: #9a603a;
9    --bs-primary-rgb: 154, 96, 58;
```

Actual source: index.html, styles.css

Verification and reflection

Recorded verification: 417/417 checks passed across 3 browser runs; 0 runtime/resource errors.

Evidence. Screenshots show the resulting pages; source screenshots use literal current code with file names and line numbers. Browser results are stored in screenshots/test-results.json.

Live verification. Recorded verification: 139/139 checks passed; 0 runtime/resource errors.

Deployment. Application commit 7539d5e was pushed to the shared main branch. GitHub Pages workflow 40739785068 completed successfully. The updated Bootstrap website returned HTTP 200 with the original Assignment 3 heading and local assets. All 139 checks passed on the published site, including all eight pages and desktop/mobile layouts, with zero JavaScript or resource errors.

DOM and styling. Read More visibility, dynamic inventory text, result counts and background colour.

Events and data. Click listeners, arrow-key focus, object methods, arrays, loops, filter and switch.

Inquiry. Invalid values, Next and Back, retained values, review, confirmation and reset.

Sound, motion and layout. Actual audio playback, repeated-click restart, transitions, offline assets and responsive pages.

Challenges and solutions. A continuation needs to keep existing layout rules while adding state changes. The scripts target small components and keep the Bootstrap markup. Form steps can hide invalid controls, so validation identifies and focuses the correct field before advancing. Repeated sound clicks are handled by restarting one player.

Reflection. Reusable functions and structured vehicle data make the added features easier to understand and change. Keeping the original pages also makes the connection between Assignments 3 and 5 clear. The inquiry remains an honest local demonstration.

Defense and submission. Every member should explain the code and practise a small change. The defense is worth 40%. Each student submits the PDF and ZIP individually and attends the practice lesson. The supplied LMS screenshot shows 29 October 2026, 14:00; check the LMS for the current deadline.